HERMES MULTI-AGENT · END-TO-END

The Memory System — Comprehensive Report

Everything done on memory, April → May 2026: the multi-tier origins, the pivots, the single-tier overhaul, the plugin patches, the LLM saga, and the 2026-05-26/27 outage recovery & quality overhaul — with current state and roadmap.

As of 2026-05-27 · supersedes the 2026-05-17 architecture brief (brings it current) · v2 `@memtensor/memos-local-plugin`

THE SYSTEM IN ONE SLIDE

What the memory does & how it’s structured

8 agents on one box, each with a private profile, sharing knowledge through one local memory plugin. Three layers, learned from experience and re-injected into every turn.

The layers

- **L1 · traces** — every turn captured (user+agent+tools), summarized, embedded.
- **L2 · policies** — induced “when X, do Y” patterns from clusters of traces.
- **L3 · skills + world-model** — crystallized callable procedures + durable environment facts.

The loop

- **Capture** → reflect/score → **reward** (LLM R_human + V backprop) → **induce** L2 → **crystallize** L3.
- **Retrieve** tier-1 skills + tier-2 policies + tier-3 traces, inject into the next prompt.
- **Isolation**: row-level `(owner_agent_kind, owner_profile_id, share_scope)` — no cubes, no API keys.

THE JOURNEY

From multi-tier orchestration to a single hardened plugin

Apr 6–8	**MemOS v1 stood up** (Qdrant + Neo4j + SQLite) under Paperclip/CEO orchestration; security audit + hardening.
Apr 20–25	**v2 migration plan** (server → local plugin); Sprint-2 gate; acceptance amended after audit.
Apr 27–28	Pivots: **v2 deprecated → revert to v1**, then **collapse the two-tier memory to a single tier**.
May 16–17	**The overhaul**: v2-only, single machine/tier/plugin; BGE-large embedder; share-scope policy; the orphan-cron root cause found & killed.
May 21	**CTO agent** added on Claude Code (peer cross-agent, no central orchestrator); dirty-episode boot fix.
May 24–25	LLM saga: DeepSeek **402** → Gemini stopgap → DeepSeek revert; **bounded capture** fixes for the cold-boot hang.
May 26–27	**Capture outage** RCA + recovery, memory-quality audit, and 6 quality fixes (this report's second half).

PART I

Foundations & the road to v2

Origins, the pivots, and the 2026-05-17 single-tier overhaul.

PHASE 1 · ORIGINS

The original multi-tier design (and why it was heavy)

- **Paperclip / CEO orchestration** — a central orchestrator routed work across agents and machines (“CEO on Tower”).
- **MemOS v1 server** — Qdrant (vectors) + Neo4j (graph) + SQLite, isolation via **per-cube API keys + URL**, sharing via **cross-cube replication** (untested).
- Operationally heavy: multiple datastores, per-cube key management, multi-machine routing, a shared API key the audit worried about.
- Extensive security audit + hardening (Apr 6–7): authorization, admin API, secret management, blind key audit.

PHASE 2 · THE PIVOTS

Three course-corrections in two weeks

- **Apr 20–25** — planned migration from the MemOS **server** (Product 1) to the **local plugin** (Product 2); Sprint-2 gate probes; acceptance criteria amended after audit triage.
- **Apr 27** — **v2 deprecated, revert to v1**: the early v2 plugin lacked operational tooling, so the audit sent us back to v1.
- **Apr 28** — **collapse two-tier → single-tier**: drop the layered server+plugin split; commit to one MemOS tier.
- **Apr 22** — hedged with a **memory-alternatives scoping** pass in case Product 2 didn't pan out.
- Net: complexity was the enemy. Every pivot pushed toward **fewer moving parts**.

PHASE 3 · THE 2026-05-17 OVERHAUL

One machine, one tier, one plugin

DECISION	BEFORE (V1 / MULTI-TIER)	AFTER (V2-ONLY)
Backend	Qdrant + Neo4j + SQLite server	Single SQLite + local ONNX, one daemon :18800
Isolation	Per-cube API keys + URL	Row-level `(agent_kind, profile_id, share_scope)`
Sharing	Cross-cube replication (untested)	`share_scope=local` rows visible across profiles, in-process
Orchestration	Paperclip / CEO, multi-machine	None — agents share via memory layers only

The April audit flagged v2 for missing tooling — so we **built the tooling** (cross-agent inspector, forensic-audit triggers, per-agent viewer, stress test) and shipped v2 in place. Paperclip/CEO + the v1 server were retired (kept on disk for rollback).

Embedder finally fixed: BGE-large

- Chose `Xenova/bge-large-en-v1.5` — **1024-dim, local ONNX** via Transformers.js (no API, no cost, no rate limit).
- Picked after a **3-way benchmark** on real content vs MiniLM (smaller/weaker) and gemini-embedding-2 (cloud, adds latency + cost + a query prefix quirk).
- Local embedding is also what makes **bridge cold-boots heavy** (~2 cores to load) — a tradeoff that resurfaces in Part III.

1024
dimensions

$0
per-embedding cost (local)

3-way
benchmark before commit

Private experience, shared knowledge

TABLE	SCOPE	WHY
traces	private	Each agent sees only its own turn-by-turn captures
episodes	private	Session boundaries are agent-local context
policies	private	L2 candidate patterns are local experiments
world_model	local	Environment + operator facts help every agent
skills	local	Crystallized procedures are the team's shared know-how (attribution per row)

Visibility rule: a row is visible IFF `share_scope ∈ {local, public, hub}` OR it matches the caller's namespace — enforced at **every retrieval**, not by trust. Per-row attribution is automatic.

The orphan cron: every 15 min, privacy leaked

- **Symptom**: `share_scope` kept flipping `private → local` on its own — looked like a plugin bug.
- **Root cause**: a **cron job every 15 min** ran `promote-memos-shares.py` — writing `local` to **every row directly via the SQLite CLI**, bypassing the plugin's namespace filter entirely.
- It lived in an **orphan Claude worktree** and was Sprint-3 scaffolding so the (now-retired) Paperclip/CEO orchestrator could read across cubes. We retired Paperclip 5 days earlier — the cron lived on.
- **Fix**: disabled in crontab (with rationale, not deleted). **Forensic-audit triggers** remain installed as a tripwire for re-emergence.

The bundled plugin has no repo — so our changes are tracked patches

Patches live in `tools/plugin-patches/`, re-applied by `postinstall-patches.sh` after any reinstall; `--check` verifies markers. Grew 13 → **22** this session.

Foundational (05-17 → 05-25)

- Shared-skill attribution (“learned by <profile>”).
- Per-request “view as <profile>” override + viewer overlay.
- Skill packager defaults to `local` share.
- Named-speaker memory summaries (multi-human).
- Chunked batch reflection + tool-call cap + bridge keepalive 10→90s (boot-hang).

This session (05-26/27)

- Bridge `session.open` timeout 30→120s.
- Failure-avoidance L2 induction + anti-patterns.
- World-model embedding-merge (dedup).
- Outcome-penalty reward (error_signatures → V).
- Crystallization bar + usage tracking.

PART II

The LLM & capture-pipeline saga

Providers, the boot-hang, and the CTO agent (May 21–25).

MODELS

Who scores, summarizes, and crystallizes

ROLE	MODEL (TODAY)	HISTORY
Embedder	BGE-large (local ONNX)	Stable since 05-17
Extraction / summary / R_human judge	DeepSeek V3 `deepseek-chat`	DeepSeek → Gemini (402 on 05-24) → DeepSeek (refunded, 05-25)
Skill crystallization / L2 induction	DeepSeek-R1 `deepseek-reasoner`	MiniMax M2.7 → NVIDIA (throttled) → DeepSeek-R1

Why DeepSeek: non-thinking (no <think> pollution), no free-tier RPM throttle — the bursty capture load needs that. Gemini was a stopgap while DeepSeek was at $0; NVIDIA-free remains a candidate **failover** for background jobs.

MAY 21–25

The cold-boot hang & the CTO agent

Bounded capture (05-25)

- Cold boot re-reflected “dirty” episodes, firing many/huge LLM calls → `malformed`/timeout → boot stalls.
- Fix: **chunk reflection** into ≤N-step calls, **cap tool_calls at 24** + hard maxTokens, heuristic orphan summaries (no LLM).
- Bridge keepalive 10→90s + close-on-failed-boot to stop process leaks.

CTO agent (05-21)

- A standalone CTO persona on **Claude Code** (not Hermes), wired to the v2 plugin as a `cto` profile.
- Re-introduced **peer** cross-agent interaction — but **no central orchestrator, no cross-machine routing**.
- This very session also runs as a captured `claude-code` profile.

PART III

The 2026-05-26/27 session

Outage recovery, memory-quality audit, and 6 quality fixes.

THE ALARM

“Memory has reset — I lost the last 3 weeks”

What it looked like

- Viewer nearly empty; overview counts “—”.
- 3D map = a sea of “Empty turn sequences”.
- Recent days seemed gone.

What was true

- **Nothing deleted** — 95k rows on disk throughout.
- The v2 store was **born 2026-05-12**; no pre-05-12 history ever lived in it.
- **Capture had stalled**; the map was a **stale snapshot** of mostly-duplicate junk.

TWO ROOT CAUSES

A re-insert loop and a CPU death-spiral

#1 Runaway re-insert

- **65,741 rows for 6 real turns** (two turns ×33,840 / ×31,318).
- Cause: `subagent.record` timed out at 30s → each retry **re-inserted with no idempotency**.
- Cost was DB bloat, **not money** — ≈ $0.0115 / 30 LLM calls.

#2 Bridge cold-boot spiral

- Each gateway bridge loads BGE-large (~2 cores); concurrent boots **starve the CPU** past the 30s timeout.
- Timeout → respawn → more loads → **29 leaked processes**.
- Band-aid scripts ran `pkill -f 'bridge.cts'` — also killing the :18800 daemon.

Stabilize → clean → harden

95,174 → 324
traces (28k+ dup re-inserts removed)

1.8 GB → 566 MB
after VACUUM

7.9 → 0.5
system load

- **Idempotency guard** — a BEFORE INSERT trigger drops a duplicate `(episode_id, turn_id)`; an episode can no longer re-inflate.
- **Raised bridge timeouts** (30→120s) + a **safe-cleanup library** replacing the dangerous `pkill` band-aids.
- **:18800 daemon → systemd unit** (Restart=always, enabled) — was a fragile manual SSH-session process.
- Consistent backup first; FTS rebuilt; integrity OK; staggered restarts.

VIEWER

Login, console & the 3D map

Fixed

- **Login** — nginx mis-routed `/api/v1/auth/login` to the dashboard; added a specific `/api/v1/ → :18800` rule.
- **Console** — mixed-content guard + auth gate before `/diag/namespace`.

New on the map

- **Memories toggle** — plot individual memory dots.
- **Reward-heat** — colour by learned value (red → grey → green).
- **Bubble drill-down** — Topic → member memories → detail.

What the data looked like underneath

- **Traces**: summaries good & speaker-attributed; ~15–25% was test/trivial noise (pruned).
- **Skills**: 41, only **4 invocations** — crystallized too eagerly, hyper-specific, never re-matched.
- **Policies**: **155 success : 1 failure**; confidence pinned at the 0.5 default.
- **World-model**: near-duplicates, under-populated.
- **Reward**: the LLM judge **is** firing — 68/102 episodes have differentiated `r_task` (−0.75…0.82) — but objective tool-failure signals were unused.

QUALITY FIXES SHIPPED

Make the memory learn better

- **Failure-avoidance induction** — L2 prompt classifies success vs failure from trace outcomes + emits anti-patterns. Fixes the 155:1 imbalance.
- **World-model embedding-merge** — semantically-identical facts merge (cosine ≥ 0.86) instead of duplicating.
- **Outcome reward penalty** — tool-error steps score −0.35 below neutral, so failures rank low + seed avoid-policies.
- **Crystallization bar + usage tracking** — a pattern must recur ≥3 episodes (was 1) to become a skill; usage now recorded on retrieval (verified 2 → 8).
- Pruned test-artifact memories, dead/duplicate skills, and a duplicate world-model.

STATUS

WHAT WORKS

🟢 Stability

- :18800 daemon healthy & under systemd (survives reboot).
- **Phase 1 complete** — boots serialized (P1.2 flock, no spiral) + non-blocking (P1.1, server up ~13s); dead-skill auto-prune (P1.3) + green stress-test (P1.4).
- No bridge leak; capture is idempotent.
- All 8 gateways active; this session captures too.

🟢 Memory & viewer

- DB clean (324 traces), FTS in sync, integrity OK.
- LLM reward judge firing → differentiated value.
- Viewer login + map heat/drill-down; graph auto-refresh succeeds again.

STATUS

WHAT DOESN’T (YET)

- **Per-gateway model loads remain** — boots are now serialized (P1.2 flock, so no spiral), but each gateway still loads its own BGE-large; the structural fix is the shared single daemon (Phase 3, P3.1).
- **Reflection can briefly blip HTTP responsiveness** — boot no longer blocks on it (P1.1: server starts in ~13s, reflection backgrounds), but Node is single-threaded so a heavy reflection burst can momentarily slow `:18800` (recovers to ~30ms). The full cure is a worker thread / the shared daemon (Phase 3).
- **Per-step verifier feedback is dormant** — episode-level LLM judge works, but fine-grained step→repair only fires on explicit markers nobody sends.
- **No human-in-the-loop signal** — reward is LLM-self-graded; no 👍/👎 channel.
- **Web stack idle-stopped** — Firecrawl/SearXNG down on-demand (2/36 stress checks, unrelated to memory).

ROADMAP

WHAT'S LEFT

Architecture (highest leverage)

- ~~Serialize bridge boots~~ — ✓ **done (P1.2, 2026-05-27)**: cross-gateway flock, one model load at a time.
- **Shared single daemon** — gateways become thin clients of :18800 instead of each loading BGE-large. **In progress (P3.1, 2026-05-27):** daemon-side capture-write HTTP routes landed (`POST /api/v1/turn/start|end`, per-request `X-As-Profile` namespace, verified isolation); remaining = per-boot adapter token + HTTP-client transport cutover behind `MEMOS_TRANSPORT`.
- ~~Background the boot-reflection~~ — ✓ **done (P1.1, 2026-05-27)**: server starts immediately; reflection backfills async.

Learning quality

- **Feed tool-failures to auto-repair** + per-step LLM verifier → finer policy repairs (cross-layer: adapter → bridge RPC → core; scheduled, tested session).
- **Explicit 👍/👎 channel** (Discord reactions). *Note: the implicit next-turn signal is already covered by the LLM R_human judge — verified 2026-05-27.*
- ~~Auto-prune dead skills~~ — ✓ **done (P1.3)**; runs as usage data accrues.

BOTTOM LINE

From a fragile multi-tier stack to a clean, hardened, self-improving memory

1	**22**	**324**	**200**
machine / tier / plugin	tracked plugin patches	clean memories (FTS in sync)	:18800 health · daemon enabled

- Decision docs: `memos-setup/learnings/` (Apr 6 → May 26) · architecture briefs in `docs/architecture/`.
- Next priority: the architecture items (serialize boots / shared daemon) to make stability structural.